

Geo Intelligence MVP

Manager Briefing Report | Architecture, AI Usage, OSM Logic and Recommendations

This report explains the Geo Intelligence MVP in plain language: what it does, the technology stack behind it, how the LLMs are actually used, how OpenStreetMap (OSM) data drives the analysis, and how the code decides which streets are Gold, Silver or Bronze for a chosen business activity. It closes with a recommendation on which paid LLMs and APIs to adopt to move from a working MVP to a production-grade decision tool.

1. What the App Does

A web application that helps an entrepreneur decide **where to open a shop**. The user supplies their city (auto-detected), a business category and a search radius. The app then:

- Pulls every nearby business, transit stop, anchor POI and residential building from OpenStreetMap.
- Groups businesses by street and scores each street for the chosen category.
- Classifies each street as **Gold**, **Silver** or **Bronze**.
- Shows the results on an interactive map and writes a market-analysis report.
- Suggests reputable property portals where the user can browse real shop listings for rent or sale.

2. Technology Stack

Layer	Technology	Why it was chosen
Frontend	Next.js 14 (App Router) + React 18	Single framework for UI + API; fast SSR; free hosting on Vercel.
Styling	Tailwind CSS	Rapid, consistent UI without writing CSS files.
Maps	Leaflet + react-leaflet	Free, no API key; works directly with OSM tiles.
Backend	Next.js API Routes (Node.js)	Lives in the same project as the frontend — no separate server needed.
Geographic data	OpenStreetMap (Overpass + Nominatim)	Free, global coverage, no API key required.
AI layer	OpenRouter free LLMs → local Ollama → template fallback	Always returns a response, even if every AI provider is down.

3. LLMs Used in the App and Their Real Purpose

The app uses a **chain of free LLMs** through OpenRouter, falling back automatically when a model is rate-limited or unavailable. If every cloud model fails, a local Ollama instance is tried, and finally a deterministic template is used so the app always produces output.

Model	Role in the chain
openai/gpt-oss-120b:free	Primary — highest-quality writer, tried first.
openai/gpt-oss-20b:free	Fast fallback when the 120b model is busy.
minimax/minimax-m2.5:free	Alternate provider for diversity.
meta-llama/llama-3.3-70b-instruct:free	Last-resort backup.
llama3.2 (local Ollama)	Offline fallback if all cloud LLMs fail.
Deterministic template	Final safety net — guarantees a response.

What each LLM call is actually used for

The LLM **never decides the ranking**. It only writes language *after* our code has done the math. Five distinct LLM calls are made, each for a different purpose:

#	Call	Purpose
1	explainStreet	Writes a 3-sentence note for each Gold/Silver/Bronze street, justifying its tier using the real numbers (competitors, transit, anchors, density).
2	generateOverallReport	Produces the 4-section executive report — Area Analysis, Market Insights, Competitor Analysis, Final Recommendation — in a Big-4 consulting tone.
3	generateRecommendationReasoning	Writes a 1-paragraph justification for each of the 3 recommended map pins, explaining why that exact spot was picked.
4	generateCompetitorInsights	Explains why existing competitors of the same category are succeeding in this area (local culture, footfall, demographics).
5	findPropertyAgencies	Returns the top 4 reputable real-estate portals for the user's city, with deep-link search URLs filtered to commercial/retail listings.

4. Is OSM an API?

OpenStreetMap (OSM) is the database, not the API. Think of it as a Wikipedia for maps — volunteers maintain global data on roads, shops, parks, transit stops, hospitals, and more. To *read* OSM data we use three free APIs built on top of it:

API	What it does	Used in our app for
Overpass API	Runs queries against the OSM database (e.g. "every café within 3 km of this point").	Finding competitors, all commercial businesses, street shapes, transit, anchors, residential buildings.
Nominatim API	Reverse geocoding — converts a lat/lon to a street/address and vice versa.	Resolving the street name of a business when its OSM tag is missing.

OSM Tile Server	Serves the actual map background images shown in Leaflet.	The grey street map you see behind the colored Gold/Silver/Bronze lines.
-----------------	---	--

Cost: \$0 — no API keys, no billing. **Limit:** These free services are shared globally and can be slow or rate-limited at peak times (10–30 seconds for a first query is normal).

5. The OSM-Powered Core — Logic of Each Part

The whole intelligence of the app comes from **four OSM queries** plus our own scoring code. Each part has one clear job:

Part A — Find direct competitors

File: lib/overpass.js → fetchCompetitors(). Maps the chosen category (e.g. coffee_shop) to OSM tags (amenity=cafe) and asks Overpass for every matching node inside the radius. Returns {id, name, lat, lon}.

Part B — Find all commercial activity

File: lib/overpass.js → fetchCommercialActivity(). Asks Overpass for every shop, café, restaurant, pharmacy and salon in the radius (not only the user's category). Drives the **density** and **variety** metrics used in the score.

Part C — Find street shapes

File: lib/overpass.js → fetchStreetGeometries(). Pulls every road polyline plus its road class (primary, secondary, residential, pedestrian...). The class feeds the road-class bonus in the score and the polyline lets us draw colored Gold/Silver/Bronze street lines on the map.

Part D — Find footfall drivers (enrichment)

File: lib/enrichment.js. One Overpass query collects three datasets at once:

- **Transit stops** — bus stops, railway stations, tram stops.
- **Anchor POIs** — malls, schools, hospitals, hotels, mosques, parks.
- **Residential buildings** — proxy for the catchment population nearby.

Part E — Group businesses by street name

File: lib/streets.js + lib/nominatim.js. For each business, first try its *addr:street* OSM tag. If missing, call Nominatim reverse geocoding (capped at 25 lookups per request to respect rate limits). Bucket every business under its street.

Part F — Score each street (no AI involved)

File: lib/scoring.js. Plain JavaScript math:

```
score = 100
  - (competitors × 18)
  + (variety × 2)
  + min(density, 30) × 1.5
  + transit bonus      (up to +45)
  + anchor bonus       (up to +48)
  + residential bonus  (up to +25)
  + road-class bonus   (-12 to +25)
```

Classification: ≥ 160 = **Gold**, 110–159 = **Silver**, < 110 = **Bronze**.

Part G — Pick the exact pin location

File: app/api/analyze/route.js → pickBestSpot(). Walks along each winning street's polyline, samples points, and at each point computes a mini-score: **positive** for nearby anchors (weighted by category — e.g. mosques weigh more for a men's salon, hospitals for a pharmacy, malls for clothing) and **negative** if a competitor is too close. The 3 final pins are forced to be at least 250 m apart so they're

not duplicates.

Part H — Orchestrate everything

File: app/api/analyze/route.js. Fires the 4 OSM queries + 1 LLM call **in parallel**, groups businesses by street, runs the scoring formula, sorts and filters, picks the top 3 pins, then fires **4 more LLM calls in parallel** for the narrative. One JSON response is returned to the frontend.

6. How a Street Becomes Gold, Silver or Bronze — Step by Step

Step	Stage	What happens
Step 1	User input	User supplies location (auto from IP), business category and radius (2 / 3 / 5 km).
Step 2	Four OSM queries in parallel	(1) all same-category competitors, (2) all commercial businesses, (3) every street polyline + road class, (4) transit stops, anchor POIs and residential buildings.
Step 3	Group businesses by street	Use the addr:street OSM tag where available; fall back to Nominatim reverse geocoding for the rest.
Step 4	Count six things per street	Competitors on-street, variety of commercial types, density, transit within 300 m, anchors within 250 m, residential buildings within 250 m. Also read the road class.
Step 5	Apply the scoring formula	100 baseline, minus competitor penalty, plus variety / density / transit / anchor / residential / road-class bonuses.
Step 6	Classify	≥160 = Gold, 110–159 = Silver, <110 = Bronze.
Step 7	Sort and filter	High to low score. Drop streets with fewer than 2 businesses, and drop unresolved "Area lat,lon" buckets.
Step 8	Pick the exact pin	Walk the winning street's polyline, score sample points by anchor proximity and competitor distance, force 250 m separation between the 3 pins.
Step 9	Narrate with LLMs	Five parallel LLM calls write the per-street notes, executive report, recommendation reasoning, competitor insights and property portals. The AI cannot change the ranking — it only describes it.

The mental model in one line: the code measures the street, the LLM describes the measurement.

7. Showing Available Shops per Activity (and City Regulations)

Scraping live shop-for-rent or shop-for-sale listings would breach the terms of service of the real-estate portals. To stay compliant, the app asks the LLM for the **top 4 reputable property portals** serving the user's city (e.g. Property Finder, Bayut, Rightmove, Zillow — depending on the city), with a **deep-link search URL** already filtered to commercial / retail / shop listings for both rent and sale. The user clicks through to the regulated portal itself, so the listings always comply with the city's property regulations and rules — because they live on the regulated portal, not inside our app.

8. Recommendation — Moving from MVP to Production

The MVP works end-to-end on free services and validates the concept. To turn it into a tool a business would **pay for and trust with real lease decisions**, the next investments should be:

8.1 Paid LLMs (for analysis quality)

- **GPT-5 / GPT-5-pro** (OpenAI) or **Claude Opus 4.7** (Anthropic) — far more accurate market reasoning, citation discipline, and consistency than the free OpenRouter models.
- **Google Gemini 2.5 Pro** — strong at structured-data extraction from real-estate listings.

8.2 Paid Location & Footfall APIs (real data, not just OSM)

- **Google Places API + Google Maps Platform** — accurate POIs, ratings, opening hours, popular-times curves.
- **Foursquare Places API** — high-quality venue data and visit-pattern analytics.
- **SafeGraph / Placer.ai** — real hourly foot-traffic counts by demographic — the same data used by major retailers.
- **Mapbox Movement / Veraset** — anonymised mobility data for trade-area analysis.

8.3 Real Property Listing Feeds

- **Property Finder API / Bayut API** (Middle East), **Zillow / Redfin** (US), **Rightmove / Zoopla** (UK) — bring live shop-for-rent and for-sale inventory *inside* our app.
- **Reonomy** or **CoStar** — commercial real-estate intelligence: rent benchmarks, owner data, deal history.

8.4 Demographics, Income and Spending Power

- **Esri Business Analyst** or **CACI Acorn** — household income, age, lifestyle segmentation at street-level granularity.
- **Experian Mosaic** — consumer-spend categories per neighbourhood.

8.5 The Five Elements of a Successful Retail Location

Retail-location success is driven by five elements the MVP currently only *approximates* via OSM. Paid APIs measure them directly:

- **Catchment** — who actually lives and works within walking / driving distance.
- **Footfall** — how many people physically pass the shopfront, by hour of day.
- **Visibility & frontage** — line of sight from the road and pedestrian flow.
- **Anchor pull** — proximity to traffic generators (malls, schools, transit hubs).
- **Competitive gravity** — distance to existing same-category players and brand strength.

Bottom Line

The MVP validates the concept at zero cost using OpenStreetMap and free LLMs. To turn it into a decision-grade product, the highest-leverage upgrade is: a **paid LLM** (GPT-5 or Claude Opus 4.7) + a **paid places & footfall API** (Google Places + Placer.ai or SafeGraph) + a **live property listing**

feed. Together they lift the recommendation from "directionally useful" to "genuinely actionable" — the threshold a business needs before signing a 5-year lease on the strength of our advice.